«Житомирська політехніка»
Факультет інформаційно-комп’ютерних технологій
Кафедра комп’ютерної інеженерії

ЗВІТ

З лабораторної роботи №2

З дисципліни «Програмування»

Тема: Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності

Варіант №7

Виконав:
Студент 1-го курсу, групи КІ-26-1
Спеціальність: 123 «Комп’ютерна інженерія»
Герус Данило Сергійович

Перевірив:
Викладач кафедри комп’ютерної інженерії
Лисенко Максим Сергійович

Житомир 2026

Мета роботи:

1. Набути практичних навичок проектування комбінованих алгоритмів із використанням операторів розгалуження (if-else , тернарного оператора) та оператора множинного вибору (switch-case)

2. Дослідити алгоритмічні особливості та практичну реалізацію циклів із передумовою (while), післяумовою (do-while) та параметричних циклів (for)

3. Опанувати чисельні методи наближеного обчислення функцій за допомогою степеневих рядів та рекурентних співвідношень.

4. Закріпити навички системного контролю версій Git: робота з файлом .gitignore , семантичні коміти за стандартом Conventional Commits.

5. Опанувати академічні стандарти підготовки технічної документації шляхом формування обов'язкової пояснювальної записки у форматі PDF.

1) Табулювання кусочно-неперервної функції (task1.c)

Розробки програму мовою С для табулювання кусочно-неперервної функції Y(X) на заданому проміжку $[X_{start};X_{end}]$ із кроком dX.

Математичний вираз кусочної функції для Варіанта 7:

$$X \leq 1: Y = \cos(X);\ 1 < X \leq 4: Y = 1/(X-1);\ X > 4: Y = 2X - 5$$

аналітичний розрахунок ОДЗ (області допустимих значень) функції:

- на $(-\infty; 1]$ функція $\cos(X)$ визначена для всіх X;
- на $(1; 4]$ вираз $1/(X-1)$ вимагає $X \neq 1$, а точка $X = 1$ в цей інтервал не входить;
- на $(4; +\infty)$ функція $2X - 5$ визначена для всіх X.

Отже, **ОДЗ: $X \in (-\infty; +\infty)$**

визначення точок розриву:

- **X = 1**: ліва границя cos(1) ≈ 0,5403, права границя lim 1/(X−1) = +∞. Це **розрив другого роду** (нескінченний).
- **X = 4**: ліва границя 1/3 ≈ 0,3333, права границя 2·4 − 5 = 3. Це **розрив першого роду** зі стрибком 3 − 1/3 = 8/3 ≈ 2,667.

Код:

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void) {
    double x_start, x_end, dx;

    // Введення параметрів англійською
    printf("Enter Xstart, Xend, dX separated by space: ");
    if (scanf("%lf %lf %lf", &x_start, &x_end, &dx) != 3) {
        fprintf(stderr, "Error: Invalid input.\n");
        return 1;
    }

    // Валідація
    if (x_start >= x_end || dx <= 0.0) {
        fprintf(stderr, "Validation error: Xstart < Xend and dX > 0 required.\n");
        return 1;
    }

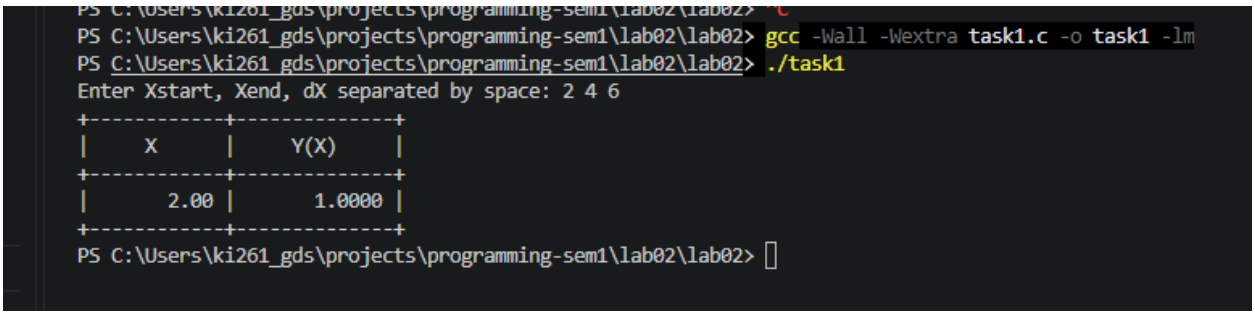
    // Шапка таблиці
    printf("+------------+--------------+\n");
    printf("|     X      |     Y(X)     |\n");
    printf("+------------+--------------+\n");

    double eps = 1e-9;
    for (double x = x_start; x <= x_end + eps; x += dx) {
        if (x <= 1.0) {
            double y = cos(x);
            printf("| %10.2lf | %12.4lf |\n", x, y);
        } else if (x <= 4.0) {
            if (fabs(x - 1.0) < eps) {
                printf("| %10.2lf | [Undefined]  |\n", x);
            } else {
                double y = 1.0 / (x - 1.0);
                printf("| %10.2lf | %12.4lf |\n", x, y);
            }
        } else {
            double y = 2.0 * x - 5.0;
            printf("| %10.2lf | %12.4lf |\n", x, y);
        }
    }

    printf("+------------+--------------+\n");

    return 0;
}
```

Повний лістинг файлу task1.c:



Опис алгоритмів та логічних рішень

1. Логіка умовних операторів та циклів

Алгоритм реалізує табулювання кусково-неперервної функції за допомогою циклу `for` та розгалуженої системи умовних операторів:

- Цикл табуляції: Ітерація відбувається від `x = x_start` до `x_end` з кроком `dx`.

- Обчислення значення функції: На кожному кроці за допомогою конструкції if ... else if ... else визначається, до якого інтервалу належить x:

  - $x \le 1.0$: обчислюється тригонометрична функція $y = \cos(x)$.

  - $1.0 < x \le 4.0$: виконується перевірка на точку розриву. Якщо $x = 1.0$, виводиться статус [Undefined] через неможливість ділення на нуль. В інших випадках обчислюється $y = 1.0 / (x - 1.0)$.

  - $x > 4.0$: обчислюється лінійна функція $y = 2.0 \cdot x - 5.0$.

2. Опис захисних заходів та валідації

Для стабільної роботи програми реалізовано кілька рівнів захисту:

1. Валідація вхідних даних: За допомогою scanf(...) != 3 перевіряється, чи введено саме три числових значення. Якщо зчитування не вдалося, програма зупиняється з помилкою.

2. Захист від зациклення: Перевірка `x_start >= x_end || dx <= 0.0` блокує виконання циклу з від'ємним або нульовим кроком `dx`, що відвертає нескінченне зациклення.

3. Обробка похибки double: Умову циклу x <= x_end + eps розширено за допомогою константи eps = 1e-9, що дозволяє включити останній вузол `x_end`, який міг би пропуститися через похибку чисел з плаваючою крапкою.

4. Безпечне порівняння та захист від ділення на нуль: Замість прямого порівняння `x == 1.0` використовується перевірка `fabs(x - 1.0) < eps`. Це дозволяє коректно виявити точку розриву та уникнути помилки ділення на нуль під час виконання.

Видає помилку якщо вказати не правильні значення.

```
PS C:\Users\ki261_gds\projects\programming-sem1\lab02> ./task1
Enter Xstart, Xend, dX separated by space: t  f d
Error: Invalid input.
PS C:\Users\ki261_gds\projects\programming-sem1\lab02>
```

2) Наближене обчислення функцій чисельним рядом

(task2.c)

Навчитися обчислювати суму нескінченного ряду за допомогою рекурентного співвідношення, використовувати цикл `while` з умовою за точністю та захист від зациклення.

$$ch(X) = 1 + \frac{X^2}{2!} + \frac{X^4}{4!} + \ldots = \Sigma \frac{X^{2n}}{(2n)!},\ n = 0, 1, 2, \ldots,\ -\infty < X < \infty$$

Ряд: ch(X) = Σ X^(2n) / (2n)!, n = 0, 1, 2, ...

a_n = X^(2n) / (2n)!

a_{n-1} = X^(2n−2) / (2n−2)!

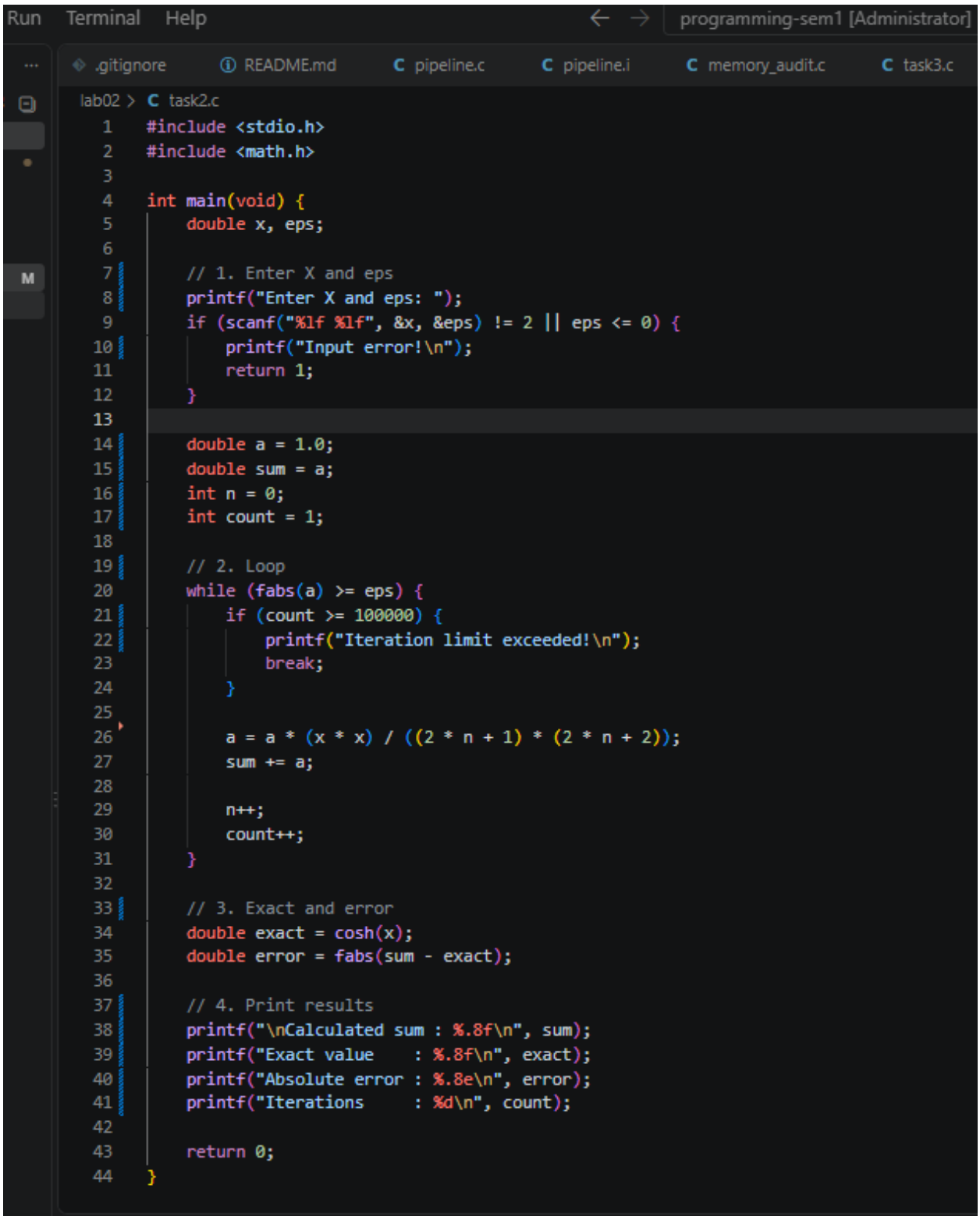
R(n, X) = a_n / a_{n-1} = [X^(2n) / (2n)!] · [(2n−2)! / X^(2n−2)]

1) Скорочення степенів:
   X^(2n) / X^(2n−2) = X^(2n − (2n−2)) = X² 2)
2) Скорочення факторіалів:
   (2n)! = (2n) · (2n−1) · (2n−2)!
   (2n−2)! / (2n)! = (2n−2)! / [(2n)(2n−1)(2n−2)!] = 1 / [2n(2n−1)]

   Кінцевий коефіцієнт множника:
   R(n, X) = X² / (2n · (2n−1))

   Отже: a_n = a_{n-1} · X² / (2n(2n−1)), a_o = 1

КОД:



Повний лістинг файлу task2.c:

```
PS C:\Users\ki261_gds\projects\programming-sem1\lab02> gcc -Wall -Wextra task2.c -o task2 -lm
PS C:\Users\ki261_gds\projects\programming-sem1\lab02> ./task2
Enter X and eps: 3 4

Calculated sum : 1.00000000
Exact value    : 10.06766200
Absolute error : 9.06766200e+00
Iterations     : 1
PS C:\Users\ki261_gds\projects\programming-sem1\lab02>
```

4. Опис алгоритмів та логічних рішень

1. Текстове пояснення логіки роботи умовних операторів та циклів

Програма здійснює обчислення нескінченного ряду для гіперболічного косинуса ch(X) = Σ X^(2n) / (2n)!, n = 0., за допомогою циклу while:

- Обчислення суми: Початковий доданок дорівнює $a_0 = 1.0$. Цикл while (fabs(a) >= eps) виконує накопичення суми sum += a, доки поточний доданок за модулем більший за точність ε.
- Рекурентна оптимізація: На кожному кроці новий доданок обчислюється через попередній за формулою $a_n = a_{n-1} \cdot X^2 / ((2n-1)(2n))$, що виключає повторний розрахунок степенів і факторіалів.
- Порівняння: Знайдена сума порівнюється з результатами стандартної функції `cosh(x)`, після чого обчислюється абсолютна похибка fabs(sum - exact).

4.2. Опис захисних заходів та валідації

- Валідація вхідних даних: Конструкція if (scanf(...) != 2 || eps <= 0) перевіряє коректність введення параметрів X та ε, перериваючи роботу при помилці.
- Захист від зациклення (Timeout Guard): Для запобігання нескінченному циклу використовується лічильник ітерацій. Якщо точність не досягається за 100 000 кроків, цикл примусово переривається командою break з виведенням попередження.
- Область збіжності: Оскільки ряд для ch(X) є збіжним для будь-якого X ∈ (−∞; +∞), додаткові обмеження на значення X не потрібні.

Видає помилку якщо вказати не правильні значення.

```
    + CategoryInfo          : ObjectNotFound: (./TASK2.String) [], CommandNotFoundExcep
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Users\ki261_gds\projects\programming-sem1\lab02> ./task2
Enter X and eps: t t
Input error!
PS C:\Users\ki261_gds\projects\programming-sem1\lab02>
```

7. Висновки

1. Швидкість збіжності ряду:

- При X ≈ 0 ряд збігається найшвидше (потрібно всього кілька ітерацій).

- При збільшенні |X| кількість ітерацій зростає через повільніше спадання перших членів ряду, але завдяки факторіалу у знаменнику ряд залишається збіжним для всіх X.

2. Ефективність рекурентного обчислення:

- Використання рекурентної формули замість прямого обчислення pow() та факторіала зводить розрахунок до кількох простих арифметичних операцій на ітерацію.

- Це суттєво підвищує швидкодію та запобігає переповненню типів даних при великих значеннях n.

3. Досягнення мети роботи:

- Засвоєно практичні навички алгоритмізації процесів табулювання та обчислення числових рядів мовою C.

- Створені програми містять усі необхідні захисні заходи (перевірка введення, ліміт ітерацій) та працюють коректно, отже мету роботи досягнуто в повному обсязі.